

Figure 4: Construction of graph edges in SDF-Sim.

3.1 Training SDF functions per object

We represent SDF $f_\theta(\mathbf{y})$ as an MLP that takes in a 3D point $\mathbf{y} \in \mathbb{R}^3$ and outputs a scalar SDF value. Learned SDFs are pre-trained separately for each object and remain fixed throughout the simulation.

Mesher To compare to the existing mesh-based baselines, we apply SDF-Sim on benchmarks where meshes are available and train the learned SDF $f_\theta(\mathbf{y})$ from a mesh. To train an SDF, we sample points in 3D space and compute the ground-truth signed distances from these points to the mesh surfaces using a classic BVH [8] algorithm. Finally, we train an MLP $f_\theta(\mathbf{y})$ to fit these distances with supervision. See more details in section D. We use the same architecture and the model size for each object shape of 8 MLP layers with 128 units each, unless otherwise stated.

Vision We use VolSDF [39] to distill an SDF from a set of images representing a 360-degree view of the outdoor garden scene first described in [4] with camera poses estimated via COLMAP [28]. See visualisation in Figure 3 and section D.2 for details.

3.2 Learned simulator

Object representation We represent the shape of the object O_i in the reference pose (centered at zero) as a learned SDF function f_{θ_i} . At a simulation step t , a rigid transformation $\mathcal{T}_i^t = (\mathbf{p}_i^t, \mathbf{R}_i^t)$ transforms the object from reference pose to the current pose in the simulation. Here, $\mathbf{p}_i^t$ is an object translation, corresponding to the object’s center of mass at timestep t ; $\mathbf{R}_i^t$ is a rotation. The task of the learned simulator is predicting the next-step transformation $\mathcal{T}_i^{t+1}$ for each object. In the following text, we omit the time index t for brevity.

To represent I objects in the input graph, we introduce *object* nodes $\mathcal{V}_O = \{\mathbf{o}_i | i = 1..I\}$, located at position $\mathbf{p}_i$, and a set of *surface* nodes $\mathcal{V}_S = \{\mathbf{n}_{ik} \in O_i | i = 1..I, k = 1..K_i\}$ on the surface of the objects, where the number of nodes K_i may differ for each object. These surface nodes $\{\mathbf{n}_{ik}\}$ move with their corresponding object according to the transformation $\mathcal{T}_i^t$. With a slight abuse of notation, we will refer to $\{\mathbf{n}_{ik}\}$ both as the node entities and their 3D position in the simulation space.

Nodes and edges within the object To construct the graph connectivity *within* an object, we follow an established line of work on learned simulators [1, 15, 25]. We connect surface nodes $\{\mathbf{n}_{ik}\}$ to their corresponding object node $\mathbf{o}_i$ using edges $\mathcal{E}_i^{\text{intra}} = \{\mathbf{e}_{\mathbf{o}_i \leftrightarrow \mathbf{n}_{ik}} | \mathbf{n}_{ik} \in O_i\}$. Thus, all the information about object motion, e.g., impulses from collision events, is propagated between the nodes via the object node $\mathbf{o}_i$. As shown by [15] we can omit the surface edges between the nodes $\{\mathbf{n}_{ik}\}$ without loss of accuracy.

In graph networks, nodes and edges are associated with feature vectors that can hold the information about the motion and the relation between the nodes. We follow the approach of [1, 15, 25] to construct a set of nodes and edge features that are necessary for simulation: To construct the node features for surface nodes $\mathbf{n}_{ik}$, we compute the finite-difference velocity estimates in the simulation space using a history of the latest three timesteps $\mathbf{v}_{ik} = (\mathbf{n}_{ik}^t - \mathbf{n}_{ik}^{t-1}, \mathbf{n}_{ik}^{t-1} - \mathbf{n}_{ik}^{t-2})$. We set node features to be $[\mathbf{v}_{ik}, \|\mathbf{v}_{ik}\|, \phi_i]$ where ϕ_i are the constant object parameters: mass, friction and restitution. We use the same procedure for the object nodes $\{\mathbf{o}_i\}$ using their positions $\mathbf{p}_i$. For intra-object edges $\mathcal{E}_i^{\text{intra}}$, we use displacement vector between the surface node position and the object center as the edge feature $\mathbf{e}_{\mathbf{o}_i \leftrightarrow \mathbf{n}_{ik}} = [\mathbf{o}_i - \mathbf{n}_{ik}, \|\mathbf{o}_i - \mathbf{n}_{ik}\|]$.

SDF-based inter-object edges. Here we introduce a new way to construct collision edges between the objects by leveraging their SDF representations (Figure 4). We design these edges such that they contain sufficient information to detect collisions, while their number remains linear in the number of nodes. This is unlike quadratic number of collision edges in mesh-based simulators.

Consider two objects O_i and O_j . For a node $\mathbf{n}_{ik}$ on O_i , we directly query the SDF f_{θ_j} of object O_j to get the distance d_{ik}^j from $\mathbf{n}_{ik}$ to the closest point on O_j . We note that unlike mesh-based approaches, this is a single test, and we do not need to calculate the distance from $\mathbf{n}_{ik}$ to *each* node/triangle on O_j . Then, if this distance d_{ik}^j is within a predefined distance threshold $\mathcal{D}$, we connect the surface node $\mathbf{n}_{ik}$ directly to the opposing *object* node $\mathbf{o}_j$ and refer to this edge as $\mathbf{e}_{\mathbf{o}_j \leftrightarrow \mathbf{n}_{ik}}$. Thus, we define the set of inter-object edges as $\mathcal{E}_{ji}^{\text{coll}} = \{\mathbf{e}_{\mathbf{o}_j \leftrightarrow \mathbf{n}_{ik}} \mid \forall \mathbf{n}_{ik} \in O_i : f_j(\mathbf{n}_{ik}) \leq \mathcal{D}\}$. This approach is different from mesh-based simulators [1, 15, 25] that connect pairs of nodes or triangles on the two surfaces.

We construct the features for collision edges $\mathbf{e}_{\mathbf{o}_j \leftrightarrow \mathbf{n}_{ik}}$ such that they contain information about potential points of collision. First, we compute the closest point $\mathbf{c}_{ik}^j$ from $\mathbf{n}_{ik} \in O_i$ to the surface of object O_j . To do so, we transform the position of $\mathbf{n}_{ik}$ into the reference space of O_j using $\mathcal{T}_j^{-1}(\mathbf{n}_{ik})$. We call an SDF function to get the distance from the node $\mathbf{n}_{ik}$ to the closest point on O_j as $d_{ik}^j = f_{\theta_j}(\mathcal{T}_j^{-1}(\mathbf{n}_{ik}))$. The closest point on the surface of O_j is then computed similarly to Eq. 1:

$$\mathbf{c}_{ik}^j = \mathbf{n}_{ik} - d_{ik}^j \mathcal{T}_j \left(\nabla f_{\theta_j}(\mathcal{T}_j^{-1}(\mathbf{n}_{ik})) \right) \quad (2)$$

Note that $\mathbf{c}_{ik}^j$ lies on the surface defined by f_{θ_j} , but does not have to coincide with any surface node and is not part of node set $\mathcal{V}$. Thus, SDFs allow us to test contact between two objects at higher fidelity, without relying on the density of the surface nodes.

Finally, we construct the features for the collision edges as follows: $\mathbf{e}_{\mathbf{o}_j \leftrightarrow \mathbf{n}_{ik}} = [\mathbf{c}_{ik}^j - \mathbf{n}_{ik}, \mathbf{c}_{ik}^j - \mathbf{o}_j, \|\mathbf{c}_{ik}^j - \mathbf{n}_{ik}\|, \|\mathbf{c}_{ik}^j - \mathbf{o}_j\|]$. These features provide information about the relative position of the closest point (a potential collision point) within the object O_j as well as its relative location to the node $\mathbf{n}_{ik}$ on the opposing object. This information is sufficient for the neural network to resolve collisions. Through message-passing over such input graph, the object node $\mathbf{o}_j$ can collect the information from all the nodes that are within a collision radius relative to the object O_j . The model also has access to the velocity and rotation history for both objects through node features, so it is able to infer how fast the node $\mathbf{n}_{ik}$ is changing its position w.r.t. O_j .

With such construction, SDF-Sim requires significantly fewer edges than mesh-based methods, because a single collision edge can test a node against an entire object surface. Asymptotically, SDF-Sim has $\mathcal{O}(I \cdot K)$ edges in the worst case, with the number of objects I being magnitudes smaller than the total number of surface nodes $K = \sum_i K_i$, instead of $\mathcal{O}(K^2)$ for mesh-based simulators. As we demonstrate below, this choice of input graph and simulator unlocks the ability to scale to very large scenes that has not been previously shown in the literature.

Graph network simulator We encode the input graph using MLPs for each node and edge type. We process the graph using 10 message passing steps as in [1, 25]. We decode the processed surface node features into an acceleration estimate $\mathbf{a}_{ik}$ for each surface node $\mathbf{n}_{ik}$, and compute a per-node position update $\hat{\mathbf{n}}_{ik}^{t+1}$ using Euler integration. Finally, next-step rigid transformations $\{\hat{\mathcal{T}}_i^{t+1}\}$ are computed from $\hat{\mathbf{n}}_{ik}^{t+1}$ using shape matching [21], following [1]. The simulator is trained on a single-step prediction task using a per-node L2 loss on the acceleration estimate $\mathbf{a}_{ik}$.

4 Results

We start by evaluating the accuracy and efficiency of SDF-Sim on small-scale simulation benchmarks, where evaluation of the baselines is feasible as well. Subsequently, we demonstrate the scaling properties of the models on scenes with an increasing number of objects. We show that SDF-Sim produces realistic rollouts of extremely large scenes with up to 1.1M nodes, which was not possible with the previous generation of learned simulators. Finally, we present ablations for how the quality of the learned SDFs affects the simulation. All simulation videos are available on the website <https://sites.google.com/view/sdf-sim>.

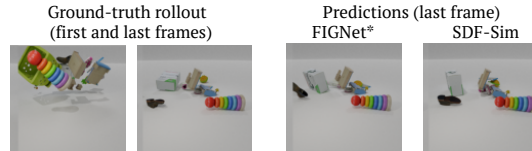


Figure 5: Comparison of the last frames of rollouts predicted on Movi-C. See more frames in Figure S7 and on the [website](https://sites.google.com/view/sdf-sim).

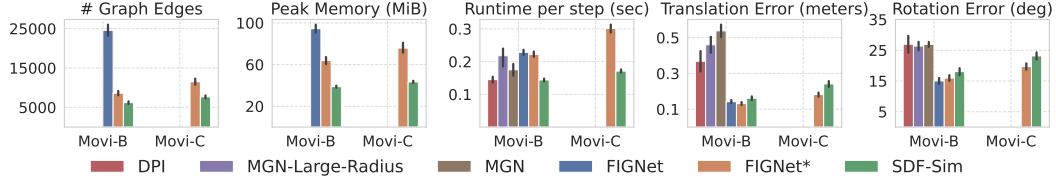


Figure 6: Accuracy, memory and runtime comparisons between the SDF-Sim model and the mesh-based baselines on the Movi-B/C benchmarks. On Movi-C, most baselines except FIGNet* run out of memory and are not shown. As “Peak Memory” we report the peak memory used by the model per single step of the simulation. DPI, MGN-Large-Radius and MGN results were reported by [1]. See Tables 3 and 4 for the exact numbers.

Kubric datasets We evaluate SDF-Sim on the benchmark Kubric datasets [13] (shared with Apache 2.0 license), consisting of simulated trajectories of rigid objects thrown towards the center of the scene. We evaluate on two difficulty tiers: Movi-B and Movi-C. Movi-B simulations contain eleven synthetic shapes (e.g., cube, torus, cow) with up to 1142 nodes each. Movi-C comprises 930 scanned real-world household objects, including hollow, flat, or otherwise non-trivial surfaces with thousands of triangles. Both Movi B/C contain only 3-10 objects per simulation. We provide more details about mesh sizes in Movi-B/C in supplement B.1.

Note that we perform many of our quantitative comparisons on small-scale Movi-B/C datasets because these domains are small enough for us to run also baseline methods to quantify efficiency and accuracy. On Movi-C we compare only to FIGNet*, because other baselines run out of memory during training due to the large number of collision edges, as reported by Allen et al. [1].

Baseline models We compare SDF-Sim to the existing state-of-the-art learned simulators for rigid-body interactions: FIGNet [1] and FIGNet* [15]. Both methods are based on graph networks. Unlike SDF-Sim, they operate directly over object meshes and use a special type of graph edges between mesh triangles. Their runtime and memory costs grow with the number and size of the object meshes used for the simulation. FIGNet* is a memory-efficient version of FIGNet that omits edges between the surface nodes of each object. In their original publication, FIGNet was demonstrated to scale up to 10 objects with a few thousands nodes each [1], while FIGNet* was tested up to a larger table scene with 40k triangles, but with only one moving object [15].

We additionally include previously reported results from Allen et al. [1] on Movi-B for the following models: DPI [14] that represents the objects as a set of disjoint particles; as well as MeshGraphNets (MGN) and MGN-Large-Radius [25] that use inter-object edges between mesh surface nodes. As reported by Allen et al. [1], DPI, MGN and MGN-Large-Radius baselines suffer from a prohibitively large size of the inputs graph and can only run on small Movi-B simulations.

4.1 Baseline comparison on small-scale datasets

First, we evaluate SDF-Sim on Movi-B/C datasets with up to 10 objects. As shown in Figure 6, our SDF-Sim uses substantially sparser graphs to represent the scenes, with 28% fewer graph edges compared to FIGNet* on Movi-B and 33% fewer on Movi-C. This translates into reduction of peak memory required to execute one step of the simulation; 39% reduction on Movi-B and 42.8% on Movi-C. The reduction in the number of edges in SDF-Sim is enabled by object-to-node collision edges that scale linearly with the number of nodes, as opposed to quadratic number of face-face collision edges in FIGNet and FIGNet* (in the worst case, see section 2). Sparser graphs in SDF-Sim also lead to lower average runtime per step of a rollout, by 36% in Movi-B and 43% in Movi-C, because the graph network performs edge updates for/over fewer edges even though the number of nodes in the graph remains the same. Figure S2(a) shows that the runtime of SDF-Sim is consistently lower than that of FIGNet* for varying sizes of the input graph. In the next section we show how these efficiency improvements translate into order-of-magnitude gains on large-scale simulations.

In terms of simulation accuracy, measured as object translation and rotation RMSE errors, SDF-Sim has substantially lower errors than previous baselines DPI, MGN and MGN-Large-Radius on Movi-B. However, in comparison to SOTA models FIGNet and FIGNet*, SDF-Sim has a slightly higher error. Note that the errors of SDF-Sim are already very low: the translation error of SDF-Sim is 0.24